

Transição para RISC-V

Explorando a ISA Moderna e o Ambiente de Simulação RARS

O Futuro é Aberto

Introdução à Filosofia e Arquitetura RISC-V

FILOSOFIA RISC-V

Design Modular

Diferente do MIPS, o RISC-V é baseado em um **Base Integer ISA** (como RV32I) fixo, com extensões opcionais.

Permite adicionar funcionalidades como multiplicação (M) ou ponto flutuante (F) sem complexidade desnecessária.

Standard Aberto

Criado na UC Berkeley em 2010 como uma ISA livre de royalties e totalmente aberta.

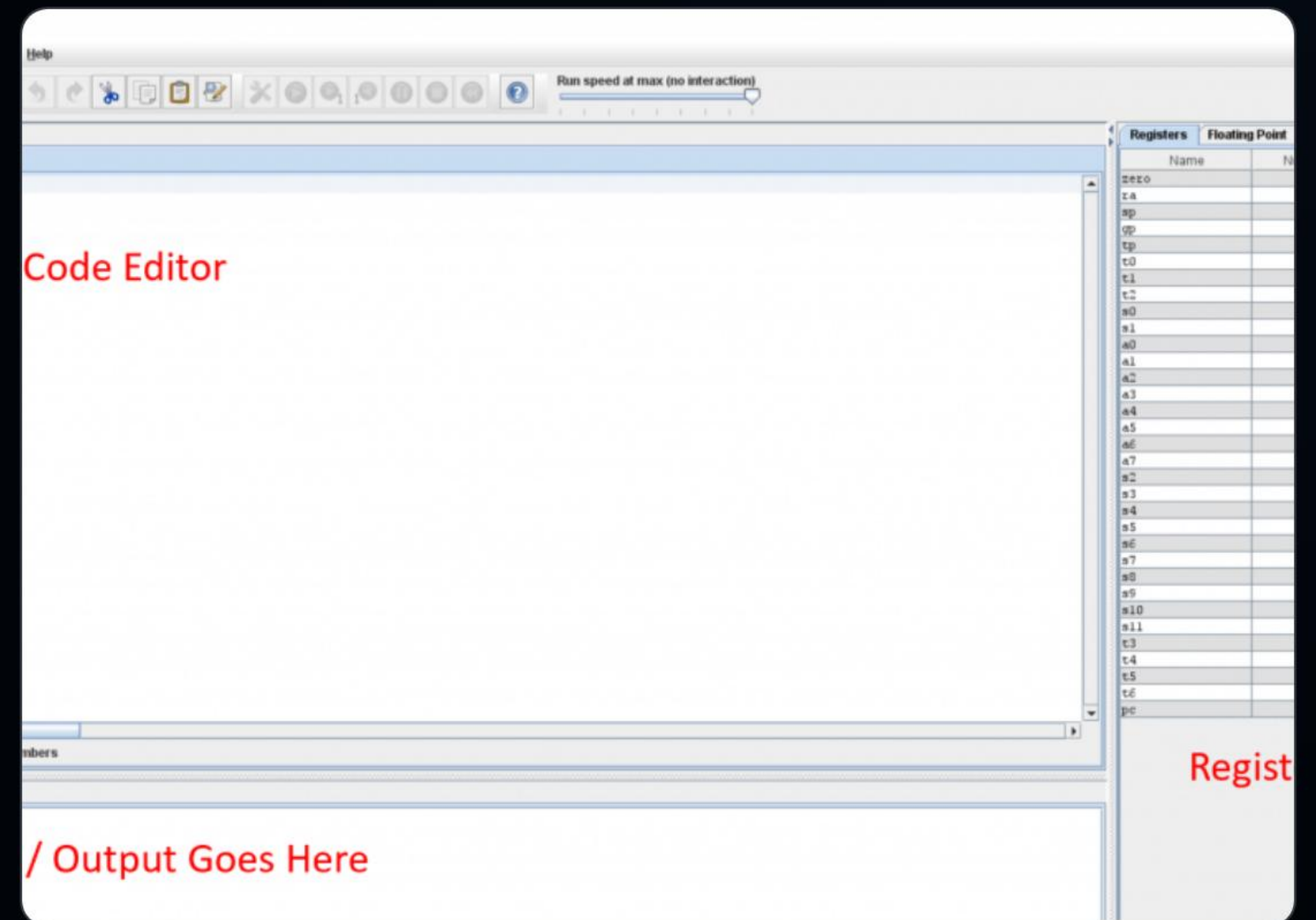
Fomenta a inovação permitindo que qualquer empresa ou acadêmico crie implementações personalizadas do hardware.

O AMBIENTE RARS

Similitude com MARS

O **RARS** (RISC-V Assembler and Runtime Simulator) é o sucessor espiritual do MARS.

- ✓ Mesma interface baseada em Java.
- ✓ Abas de Edição e Execução idênticas.
- ✓ Monitoramento de registradores em tempo real.
- ✓ Suporte total a chamadas de sistema (Ecalls).



BANCO DE REGISTRADORES

ABI Name	Register	MIPS Equivalent	Description
zero	x0	\$zero	Hardwired Zero
ra	x1	\$ra	Return Address
sp	x2	\$sp	Stack Pointer
a0 - a7	x10 - x17	\$a0 - \$a3	Function Arguments / Return
t0 - t6	x5 - x7, x28 - x31	\$t0 - \$t9	Temporaries

INSTRUÇÕES ARITMÉTICAS

- +** **Soma R-Type:** `add rd, rs1, rs2`. O resultado de $rs1 + rs2$ é armazenado em rd.
- **Subtração:** `sub rd, rs1, rs2`. Note que não existe uma instrução `subi` no RISC-V.
- ⚡** **Imediato:** `addi rd, rs1, imm`. Para subtrair constantes, utilizamos `addi` com valor negativo.
- </>** **Sintaxe Limpa:** A ordem dos operandos permanece consistente (Destino, Origem 1, Origem 2/Imediato).

ACESSO À MEMÓRIA RAM

Load Word (lw)

```
lw rd, offset(rs1)
```

Carrega 32 bits da memória no endereço **rs1** + **offset** para o registrador **rd**.

O offset é um imediato de 12 bits com sinal.

Store Word (sw)

```
sw rs2, offset(rs1)
```

Armazena o conteúdo de 32 bits de **rs2** na memória no endereço calculado **rs1** + **offset**.

Mantém a lógica clássica da arquitetura Load-Store.

ESTRUTURAS DE CONTROLE



Branches

`beq`, `bne`, `blt`, `bge`

Realizam a comparação direta entre dois registradores no próprio branch.



Loops

Implementados com branches para o topo do bloco e jumps para o fim.

Simplificação: Não há *delay slot* como em algumas versões MIPS.



Jumps

`jal rd, label` e `jalr rd, offset(rs1)`

Usados para chamadas de função e retornos (`ret` é pseudo-instrução).

ECALL VS SYSCALL

a7

Código da Chamada

Instrução Ecall

No MIPS, usávamos `$v0` e a instrução `syscall`.

No RISC-V, o código da operação vai no registrador **a7 (x17)** e a instrução é **ecall**.

Os argumentos da chamada (como strings ou inteiros) devem ser carregados de **a0 a a6**.

MIPS VS RISC-V: REGISTRADORES

Registradores MIPS	31 GPRs (\$1-\$31)
Registradores RISC-V	32 GPRs (x0-x31)

O RISC-V expande o banco de registradores e simplifica a nomenclatura para fins acadêmicos e comerciais.



Do Algoritmo ao Código

A transição de MIPS para RISC-V no RARS é fluida. O segredo está em entender o mapeamento dos registradores ABI e a mudança sutil em instruções como `ecall`.

Nesta etapa, focaremos em traduzir estruturas básicas de C para Assembly RV32I.